CHAPTER 1: Architecture design

1. **System Architecture Diagram and Data flow diagram:**

System Architecture Diagram

```
Clients
(Web / Mobile / API users)
        |
        v
+-----------------------+
|      API Gateway      |
|-----------------------|
| - Tenant validation   |
| - Rate limiting       |
| - Routing             |
+-----------------------+
        |
        v
 +-------------------+        +--------------------+
 |  Search Service   |        | Document Service   |
 |-------------------|        |--------------------|
| - Tenant-aware    |        | - Tenant-aware CRUD|
|   search          |        | - Write to MySQL   |
| - Query parsing   |        | - Index Elasticsearch|
| - Cache lookup    |        | - Invalidate Redis |
 +-------------------+        +--------------------+
        |                              |
        v                              v
+--------------------+          +--------------------+
|      Redis Cache   |          |       MySQL        |
|--------------------|          |--------------------|
| - Query cache      |          | - Source of truth  |
| - Tenant keys      |          | - Stores documents |
+--------------------+          | - Tenant partition |
        |                       +--------------------+
        v
+--------------------+
|   Elasticsearch    |
|--------------------|
| - Full-text search |
| - BM25 ranking     |
| - Tenant filter    |
+--------------------+
```

Data flow diagram.

```
Data flow diagram.

Client
  |
  v
API Gateway
  |
  +----------------------------+
  |                            |
  v                            v
Document Service           Search Service

(INDEXING FLOW)             (SEARCH FLOW)
1. Validate tenant context  1. Validate tenant context
2. Store in MySQL           2. Check Redis Cache
   (source of truth)           |
3. Index into Elasticsearch    |-- Cache HIT --> Return results to user
4. Invalidate Redis cache      |-- Cache MISS ->
                               Query Elasticsearch
                               (tenant_id filter)
                               Store result in Redis
                               Return results to user

Deletes:
- Remove tenant-scoped records from MySQL
- Delete corresponding documents from Elasticsearch
- Invalidate cached search data for the tenant in Redis
```

2. **Database/storage strategy:**
   a. **MySQL:**
      i. Source of truth for documents
      ii. Strong consistency for CRUD operations
      iii. Supports transactional guarantees
      iv. Ensures durability and recovery
   b. **Elasticsearch:**
      i. Optimized for full-text search
      ii. Inverted indexing Relevance ranking, and Fast query execution
      iii. Stores denormalized document data for search efficiency
   c. **Redis:**
      i. Caches frequent search queries
      ii. Reduces Elasticsearch load
      iii. Tenant-aware keys ensure isolation
      iv. **Example**: t1:search:distributed systems
3. **API design with key endpoints and contract examples:**

   Core Endpoints:

   - POST /documents → Index document
   - GET /search → Full-text search (tenant-aware)

- GET /documents/{id} → Fetch document
- DELETE /documents/{id} → Delete document
- GET /health → Service health

Tenant Handling:
- Primary: x-tenant-id header
- Fallback: tenant query param

Response Format:
- Standard JSON envelope (success/error)

4. **Consistency model and trade-offs:**
    a. MySQL is the source of truth for all job postings, and Elasticsearch is the search index. When a document is created, the API writes it to MySQL and then indexes it in Elasticsearch in the same request. If Elasticsearch indexing fails during creation, the request fails so the two systems do not drift apart.
    b. Elasticsearch acts as an eventually consistent read model. Writes are synchronous in this prototype, so data is immediately searchable after a successful request. However, Elasticsearch is still fundamentally an eventually consistent system due to its distributed nature (refresh intervals, shard replication). So while the write path is synchronously coordinated, the read model is logically eventually consistent.
    c. I did not add a message queue in this prototype because it would add extra operational complexity for a small assessment project. Synchronous indexing keeps the flow easier to understand and keeps the code modular. The trade-off is that writes depend on Elasticsearch being available, and the system has less buffering or retry protection than a queue-based design.
5. **Caching strategy across different layers:**
    a. Redis is used to cache repeated search queries. The cache is tenant-aware, so one tenant's search results are never reused for another tenant. When a user searches the same query again, the API can return the cached response instead of querying Elasticsearch again.
    b. Cache keys include a tenant cache version, for example: tenant-a:search:v3:distributed. A single Redis counter is maintained per tenant using a key such as tenant:tenant-a:cacheVersion. When a job is created or deleted, that version is incremented. New searches automatically use the new version, and older cache entries become stale without needing to delete them one by one.
    c. This approach keeps search fast, preserves tenant isolation, and makes cache invalidation simple. TTL is still applied so old cache entries are eventually removed from Redis.

6. **Message queue usage for asynchronous operations:**
    a. I have not added a message queue in this prototype, as setting up something like SQS would add extra infrastructure and complexity for a small assessment. Synchronous indexing keeps the flow simple and easier to understand.
    b. In a production system, a queue is important. Writing to MySQL is fast, but Elasticsearch indexing can be slower or fail. By publishing indexing events to a queue, the API remains fast and failures can be retried, improving reliability.
    c. In a previous system I worked with, synchronous indexing led to inconsistencies in Elasticsearch during failures. Moving to a queue-based approach resolved this.
    d. Amazon SQS is a good fit here because it is simple, fully managed, and sufficient for background tasks like indexing. Kafka would be more suitable for high-throughput event streams or replay use cases, while RabbitMQ fits more complex routing scenarios.

7. **Multi-tenancy approach and data isolation strategy:**
   The system uses shared infrastructure with logical separation based on tenant id. Each request will require a tenant_id across ALL layers to ensure strict tenant based data separation. This will scale well to a large

number of tenants, is efficient use of shared infrastructure, and is simpler to manage compared to per-tenant resources. Tenant isolation is achieved by including tenant_id in:

- **Elasticsearch queries** → used as a filter to restrict search results
- **Redis cache keys** → ensures cached results are tenant-specific
- **MySQL queries** → ensures only tenant-owned data is accessed

Examples:
1. Redis key: t1:search:distributed
2. Elasticsearch filter: { "term": { "tenant_id": "t1" }}

A consideration: In scenarios where data isolation is extremely critical, an alternative approach is to use separate Elasticsearch indices per tenant. It would allow strong physical isolation, no risk of missing tenant filters, and simpler query logic. But it does not scale well with many tenants and would have higher operational overhead and higher memory usage.

CHAPTER 2: Production readiness analysis
Document what would be required to make this production-ready:

1. **Scalability**:
    a. The current API is stateless, so it can run as multiple replicas behind a load balancer
    b. Use MySQL indexes, read replicas, and partitioning if needed
    c. Scale Elasticsearch with proper shard and replica planning
    d. Autoscale on CPU, memory, and request rate
    e. The current in-memory rate limiter works only per instance and does not scale horizontally. In production, this should be moved to a distributed system such as Redis-based rate limiting or enforced at the API Gateway level to ensure consistent limits across all instances.
2. **Resilience**:
    a. The create path already rolls back the MySQL insert if Elasticsearch indexing fails, which prevents source-of-truth drift
    b. a queue-based indexing layer should be added later
    c. The health endpoint already checks MySQL, Elasticsearch, and Redis, which is a good base for readiness probes
    d. Timeouts and retries should be added around MySQL, Redis, and Elasticsearch calls so a slow dependency does not hang requests
3. **Security**:
    a. The current prototype only enforces tenant scoping through x-tenant-id or tenant query params.
    b. There is no real authentication or authorization layer yet, so production would need JWT or session-based auth with tenant-aware claims
    c. In production, TLS should be enabled everywhere.
    d. Input validation is already present through class-validator DTOs, which is a good baseline.
    e. Secrets like DB passwords and ES credentials should move to the environment or secret storage.
4. **Observability**:
    a. Production should add structured logs for create, search, delete, and cache hit/miss paths.
    b. Once queueing is added, log message processing states too
    c. Right now the app has error handling, but not centralized request logging.
5. **Performance**:
    a. Redis currently caches repeated search queries for 15 seconds, which helps reduce repeated Elasticsearch hits.
    b. The current in-memory rate limiter is simple but not production-grade under horizontal scale.
    c. Performance testing should focus on search latency, cache hit rate, and the cost of synchronous ES indexing on writes.
6. **Operations**:
    a. The current docker compose setup is good for local development and demos.
    b. Production should move to managed MySQL, managed Elasticsearch, and managed Redis.
    c. Database schema changes should be handled with real migrations instead of startup-time setup.

   d. Backup and restore procedures should cover MySQL and Elasticsearch separately, since MySQL is the source of truth and Elasticsearch is derived.
7. **SLA Considerations**:
   a. To reach 99.95% availability, the API should run with multiple replicas and health checks.
   b. MySQL should have replication and failover.
   c. Elasticsearch should have replicas and reindex/recovery planning.
   d. Redis should be highly available because it affects both caching and, potentially later, distributed rate limiting.
   e. Deploy services across multiple availability zones to avoid single points of failure

---

CHAPTER 3: Experience showcase

1. **A similar distributed system I have built and its scale/impact:**

   At my previous company in the HR Tech domain, I worked on APIs that interacted with an Applicant Tracking System that supported full-text search across jobs and candidate profiles. The system handled thousands of job records and around a million candidate profiles, so search reliability and indexing consistency were important. I was not on the core search team, but by working on the surrounding APIs I gained a good understanding of the end-to-end flow.

   In the original setup, document changes were written to MySQL and then pushed directly to Elasticsearch. This created production issues when failures happened midway through the process, which led to missing or partially updated search data. To improve reliability, the team introduced SQS between MySQL and Elasticsearch. Changes were first written to the database, then queued, and processed asynchronously by indexing workers. This made the system more resilient because retries were possible and indexing failures no longer directly affected the user-facing write path. The team also stored SQS event and processing states in MongoDB for debugging and operational visibility. For multi-tenancy, account-based Elasticsearch indices were used to isolate tenant data.

2. **A performance optimization that resulted in significant improvements:**

   Let me give a bit of context. In our product, a recruiter pipeline had 5 fixed stages like sourcing, screening, interview, offer, and hire, but each stage could also have multiple substages depending on the organization. We had a reporting model built by a consulting team, and as we onboarded more customers, it started breaking in production. At that time I was mainly integrating these reports through Node.js, but my CTO asked me to look into the Power BI model itself.

   The main issue was in one of our key visuals, a matrix that allowed users to toggle between stage-level and substage-level data. To support that toggle, the model was duplicating the dataset, once for stages and once for substages, which was increasing the data size and refresh time. After confirming with the product team that we could split the feature into two visuals, I changed the model to avoid that duplication and instead derive both views from a single structured dataset.

   Due to this (and some other restructuring), refresh time dropped from around 20 minutes to around 5 minutes, and the model scaled from failing around 300k rows to handling 1M+ rows reliably. It was a good lesson for me that my job is not only to write efficient code, but also to understand the product.

3. **A critical production incident you resolved in a distributed system:**

   On a Monday morning during standup, I got pulled into a call by the CTO. The CEO and Customer Success Lead were already there, so it was clearly serious. The analytics system, which I had recently taken ownership of, was down. What had happened was that a new customer had just been onboarded, and many of their recruiters suddenly got access to the reporting module. We were running on a basic Power BI setup with no autoscaling, so the system could not handle that spike. To fix things immediately, I manually scaled up the Power BI capacity so customers could start using the system again. That stabilized the system right away.

After that, over the next couple of weeks, I worked on setting up autoscaling based on usage patterns and some basic triggers. Until that was fully in place, I kept monitoring usage and scaling manually whenever needed. After implementing autoscaling, we did not see similar incidents again.

This was one of those moments where I really understood the difference between building something and owning it in production. It is not just about making it work; it is about making sure it does not break when real users hit it.

4. **An architectural decision you made that balanced competing concerns:**
   This was in the context of the Power BI analytics system I built at Talent Recruit. We had a very lean data model on the Power BI side to keep refresh costs low and resource usage minimal. For most of the reports I had built, the required calculations could be handled in DAX, and that performed well.

   Then the product team asked for a more complicated ageing report. Computing the ageing values and dates directly in Power BI would be expensive. I could have added extra columns to the model and made the report faster, but that would have slowed down the entire refresh process.

   So I had to choose between making one report slow or making the whole refresh slow. I presented the trade-off to management and suggested that we keep the model lean and accept a slightly slower report instead. That kept refresh costs under control while still meeting the business requirement.

CHAPTER 4: Brief note on AI tool usage:

For this project, I used AI tools to speed up development while keeping the implementation structured and consistent. Since I had recently gone through system design concepts, I first combined my notes into a single reference document using Codex, which helped me keep the terminology and approach aligned while building the project.

While implementing the system, I did not try to generate everything at once. I broke the work into smaller parts such as APIs, document indexing, search, and caching, and worked through them step by step with AI assistance. I found that starting a project completely from scratch with AI was not always reliable, so I used an existing backend project as a reference and followed a similar structure for this implementation.

I used AI to move faster during development, but I still kept control over the overall design and final decisions for the system. I asked it to first propose a detailed plan, then I reviewed and approved the structure before allowing it to generate code.